

BIOS INSPECTOR: FERRAMENTA ORIENTADA A OBJETOS PARA ANÁLISE ESTÁTICA E DINÂMICA DE MÓDULOS UEFI

Gelson José de Almeida Filho
gelsonfilho@alunos.utfpr.edu.br

Disciplina: **Programação Avançada** – Prof. Dr. Robson R. Linhares
Departamento Acadêmico de Informática – **DAINF** - Campus Curitiba
Universidade Tecnológica Federal do Paraná - UTFPR
Avenida Sete de Setembro, 3165 - Curitiba/PR, Brasil - CEP 80230-901

Resumo — A análise manual de imagens de BIOS UEFI torna-se inviável em ambientes com muitas versões de firmware, pois exige inspeção repetitiva de dezenas de módulos. Este trabalho apresenta o BIOS Inspector, ferramenta em C++ orientada a objetos que automatiza a extração, organização e análise estática de módulos UEFI a partir de uma imagem de firmware, com etapa opcional de análise dinâmica via Qiling. O fluxo utiliza o CHIPSEC para decodificar a flash e o UEFIEExtract para estruturar os binários, em seguida aplica três fases principais: análise PE COFF com coleta de metadados, seções, entropia e hashes SHA 256, extração de strings ASCII e UTF 16 com geração de relatórios por módulo e casamento de padrões sensíveis, tanto de cadeias de texto quanto de GUIDs, com base em listas configuráveis. A arquitetura emprega conceitos de Programação Avançada como classes especializadas, concorrência com pool de threads e padrão DAO para persistir manifest JSON e relatórios em TXT e CSV. O resultado é uma ferramenta de linha de comando que consolida o estado dos módulos analisados e apoia revisões de segurança de firmware em escala.

Palavras-chave: UEFI, firmware, C++, análise estática, análise dinâmica, segurança de BIOS.

Abstract — Manual inspection of UEFI BIOS images becomes impractical in environments with many firmware versions, since it requires repetitive analysis of dozens of modules. This project presents BIOS Inspector, an object oriented C++ tool that automates the extraction, organization and static analysis of UEFI modules from a firmware image, with an optional dynamic analysis phase using Qiling. The workflow leverages CHIPSEC to decode the flash and UEFIEExtract to structure the binaries, then applies three main phases: PE COFF analysis collecting metadata, section information, entropy and SHA 256 hashes, ASCII and UTF 16 string extraction with per module reports, and sensitive pattern matching for both strings and GUIDs based on configurable lists. The architecture applies Advanced Programming concepts such as specialized classes, concurrency with a thread pool and the DAO pattern to persist a JSON manifest and TXT or CSV reports. The resulting command line tool summarizes the state of each analyzed module and supports large scale firmware security reviews.

Keywords: UEFI, firmware, C plus plus, static analysis, dynamic analysis, BIOS security.

INTRODUÇÃO

Este projeto se insere na disciplina de Programação Avançada do PPGCA UTFPR, tendo como objetivo consolidar conceitos de orientação a objetos em C++ por meio do desenvolvimento de um sistema de informação. Busca-se exercitar práticas de engenharia de software, incluindo uso de herança, polimorfismo, STL, concorrência, padrão DAO e tratamento de exceções, produzindo ao final um artefato funcional, reutilizável e bem documentado.

O objeto de estudo é o BIOS Inspector, uma aplicação de linha de comando destinada à análise de firmwares UEFI. A BIOS Basic Input Output System, atualmente implementada como

UEFI Unified Extensible Firmware Interface, é composta por múltiplos módulos executáveis no formato PE COFF Portable Executable, Common Object File Format, cada qual identificado por um GUID Globally Unique Identifier. Esses módulos são responsáveis por inicializar o hardware e preparar o ambiente para o sistema operacional, sendo críticos para a segurança e integridade do dispositivo.

Auditar esses sistemas é uma tarefa complexa, pois geralmente não se tem acesso ao código fonte dos módulos, apenas ao binário final recebido por integradores ou fabricantes. O BIOS Inspector automatiza esse processo ao integrar CHIPSEC e UEFIEExtract para extrair os módulos .efi de uma imagem de firmware e, a partir deles, executar três fases principais: análise PE COFF com coleta de metadados, seções, entropia e hashes SHA 256, extração de strings ASCII e UTF 16 com geração de relatórios por módulo e casamento de padrões sensíveis de strings e GUIDs com base em listas configuráveis. Opcionalmente, cada módulo pode ser executado em ambiente controlado via Qiling, com timeout configurável e geração de logs específicos. Todos os resultados são consolidados por meio de classes DAO em um manifest JSON e em relatórios TXT e CSV, que facilitam a inspeção e filtragem pelo usuário em atividades de revisão de segurança de firmware em escala.

O método adotado segue o ciclo clássico de engenharia de software, em versão simplificada, envolvendo definição dos requisitos funcionais e não funcionais, modelagem por UML com ênfase em diagrama de classes detalhado e versão amigável, além de diagrama de sequência, implementação em C++ orientado a objetos com uso de STL, pool de threads e padrão DAO para persistência, e testes de uso por meio da execução da ferramenta sobre diferentes imagens de firmware e análise dos artefatos gerados. Nas seções seguintes são apresentados o contexto e os requisitos do sistema, a modelagem em UML, os principais aspectos de implementação, a interação via linha de comando e os resultados obtidos, seguidos das conclusões sobre o projeto desenvolvido na disciplina.

EXPLICAÇÃO DO SOFTWARE

Esta seção descreve o BIOS Inspector do ponto de vista de quem utiliza a ferramenta, isto é, pelas funcionalidades e pelos artefatos que o usuário enxerga na prática, sem entrar em detalhes de implementação.

- Entrada e escopo de uso: O uso típico começa com a indicação de um arquivo de firmware (firmware.bin), obtido a partir da leitura da flash de um equipamento. A partir desse único arquivo de entrada, o BIOS Inspector produz toda a estrutura de análise na pasta de trabalho definida pelo usuário, com subpastas organizadas para módulos, strings, padrões sensíveis, relatórios e logs de execução. Esse fluxo geral, da imagem de firmware até os diretórios de saída, está ilustrado na Figura 1, com o firmware.bin à esquerda e as pastas outputs à direita.

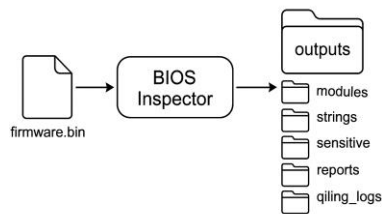


Figura 1. Visão geral do fluxo de uso do BIOS Inspector

- **Extração automática de módulos UEFI:** Depois que o usuário chama a ferramenta, o BIOS Inspector integra o CHIPSEC e o UEFIEExtract de forma transparente. O usuário não precisa interagir diretamente com esses utilitários, apenas acompanha o progresso pelo terminal. Ao final dessa etapa, os módulos UEFI presentes na imagem de firmware são gravados em formato .efi dentro da pasta outputs\modules. Na Figura 2 é possível ver a tela do terminal exibindo o progresso de extração e na Figura 3 a árvore de pastas resultante (outputs do chipsec, do UEFIEExtract e demais outputs do BIOSInspector).

```

PS C:\Users\gelo\Desktop\material_aula_prog_mestrado\trabalho_final\BIOSInspector> .\BIOSInspector.exe
[cli] Firmware: "C:\Users\gelo\Desktop\material_aula_prog_mestrado\trabalho_final\BIOSInspector\inputs\Firmware.bin"
[cli] Workspace: "C:\Users\gelo\Desktop\material_aula_prog_mestrado\trabalho_final\BIOSInspector\outputs"
[cli] Threads: 4
[cli] Qiling: QTL
[cli] Timeout: 10 s
[cli] min ASCII: 4
[cli] min UTF16: 4

==== FASE 1: CHIPSEC ====
[chipsec] Executando CHIPSEC...
[OD] python "C:\Users\gelo\Desktop\material_aula_prog_mestrado\trabalho_final\BIOSInspector\resources\chipsec-1.13.17\chipsec_util.py" uefi decode "C:\Users\gelo\Desktop\material_aula_prog_mestrado\trabalho_final\BIOSInspector\inputs\Firmware.bin"
=====
##
## CHIPSEC: Platform Hardware Security Assessment Framework
##
CHIPSEC Version : 1.13.17
CHIPSEC Arguments: uefi decode C:\Users\gelo\Desktop\material_aula_prog_mestrado\trabalho_final\BIOSInspector\inputs\Firmware.bin

[CHIPSEC] OS : Windows 11 10.0.22H2 #8064
[CHIPSEC] Python : 3.12.12 (64-bit) - enabled GIL
[CHIPSEC] helper : NoneHelper
[CHIPSEC] Platform: Unrecognized Platform
[CHIPSEC] GUID: FFFF
[CHIPSEC] VID: FFFF
[CHIPSEC] DID: FFFF
[CHIPSEC] RID: FF

==== FASE 2: UEFIEExtract ====
[>] Navego: "C:\Users\gelo\Desktop\material_aula_prog_mestrado\trabalho_final\BIOSInspector\outputs\Firmware.bin.dump" -> "C:\Users\gelo\Desktop\material_aula_prog_mestrado\trabalho_final\BIOSInspector\outputs\uefifreeextract_outputs\Firmware.bin.dump"
[>] Navego: "C:\Users\gelo\Desktop\material_aula_prog_mestrado\trabalho_final\BIOSInspector\inputs\Firmware.bin.guids.csv" -> "C:\Users\gelo\Desktop\material_aula_prog_mestrado\trabalho_final\BIOSInspector\outputs\uefifreeextract_outputs\Firmware.bin.guids.csv"
[>] Navego: "C:\Users\gelo\Desktop\material_aula_prog_mestrado\trabalho_final\BIOSInspector\outputs\Firmware.bin.report.txt" -> "C:\Users\gelo\Desktop\material_aula_prog_mestrado\trabalho_final\BIOSInspector\outputs\uefifreeextract_outputs\Firmware.bin.report.txt"
[Cleanup] Limpando artefatos temporários em: "C:\Users\gelo\Desktop\material_aula_prog_mestrado\trabalho_final\BIOSInspector\inputs"
  
```

Figura 2. Extração dos módulos de uma BIOS

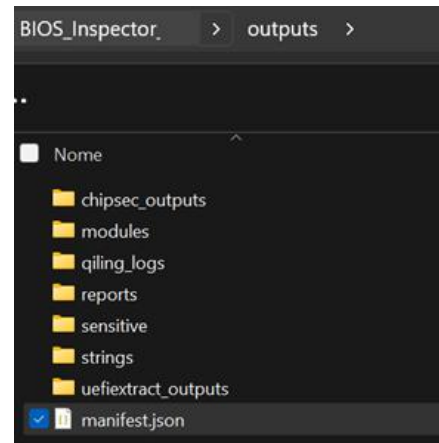


Figura 3. Estrutura de outputs

- **Visualização de métricas por módulo:** Para cada módulo .efi encontrado, o BIOS Inspector calcula e registra métricas que ajudam na triagem, como tamanho em bytes, arquitetura inferida a partir do cabeçalho PE COFF, quantidade de seções e hashes SHA 256 tanto para o arquivo completo quanto para cada seção. Essas informações ficam resumidas em um arquivo CSV chamado modules_summary.csv, armazenado em outputs\reports. O usuário pode abrir esse CSV em uma planilha e filtrar por colunas como tamanho, validade do PE, quantidade de strings e quantidade de ocorrências sensíveis. A Figura 4 destaca como os módulos aparecem listados para inspeção, neste caso, como CSV. O software também gera um relatório textual /outputs/reports/modules_report.txt, que apresenta os módulos em blocos por arquivo indicando tamanho, hash, número de strings e status de execução.

```

C:\Users> gelso > Desktop > materiad_aula_prog_mestrado > trabalho_final > BIOS_Inspector_v2 > outputs > reports > modules_summary.csv > data
1 path,fileSize,isValidPe,fileSha256,totalStrings,totalSensitive
2 "...\\AcpiTableDxe.efi",18368,1,"8906872678E568B3F9BF950361A76E47056E931EE58986D17C687368952D29B2",142,0
3 "...\\AmdSevDxe.efi",7552,1,"96AC2EE3AF72C3F9C9F8209A83DD85668B23BB5C605936CD91DB6C1044A477E8",55,0
4 "...\\ArpDxe.efi",11968,1,"38D356F0CE2ACCE2D6444F24077A9883C087D9503A24506095E2A425ED77DDC",118,0
5 "...\\AtaAtapiPassThruDxe.efi",27520,1,"9074F80728731FEF425DF84EA4AC8C52BDDDAF5CEA5F702D8DB85E50D5DD4A3E",352,0
6 "...\\AtaBusDxe.efi",13504,1,"FF8A001D83C70F3CA24F396310CDE89926A289F76279E66A1BA82840F166BA5A",141,0
7 "...\\BdsDxe.efi",78080,1,"EDFE5350763E4A8A4AE89CF6151915A810F9F23EB85DF4213D1E51346875D149",707,1
8 "...\\BootGraphicsResourceTableDxe.efi",3136,1,"A8087997BEF8A9D9F32A1AE4886BA419175FED6F12870E667B68416C35C3176",16
9 "...\\BootManagerMenuApp.efi",45824,1,"24F756D9461B366E7A02E27D00CD4EF67BAF4A091EBDC0A0D899F841236C0195",385,1
10 "...\\BootScriptExecutorDxe.efi",70528,1,"BAE83E484BCFCDACB28674279A3EC704CD2CDE448CBE19F88D0DE2AC0DADC865",295,2
11 "...\\CapsuleRuntimeDxe.efi",12288,1,"E80E42725E6D5788AE27F7899901CE26BF81396970B3A0D811EB081C3F96A256",10,0

```

Figura 4. Trecho do arquivo modules_summary.csv gerado

- Descoberta de strings e padrões sensíveis:** Outra funcionalidade percebida diretamente pelo usuário é a extração de strings. Para cada módulo analisado, o BIOS Inspector grava um arquivo de texto na pasta outputs\strings com o nome do módulo seguido da extensão .strings.txt. Em cada arquivo o usuário visualiza as strings ASCII e UTF 16 encontradas, acompanhadas do deslocamento dentro do binário e da indicação do tipo de codificação. Isso facilita localizar mensagens, caminhos, nomes de funções ou outros textos relevantes presentes na BIOS. Em paralelo, o usuário pode configurar listas de termos sensíveis e GUIDs em arquivos BlackListStrings.txt e BlackListGUIDs.txt. Durante a análise, o BIOS Inspector procura esses termos nas strings extraídas e nos GUIDs associados aos módulos. Quando encontra correspondências, registra esses achados em arquivos .sensitive.txt dentro da pasta outputs\sensitive. Nesses relatórios aparecem campos como tipo da correspondência string ou GUID, codificação e posição aproximada (offset hexadecimal) no módulo. Um recorte de um desses arquivos, mostrando algumas correspondências encontradas, pode ser visto na Figura 5.

```

BootScriptExecutorDxe.efi.sensitive.txt
outputs > sensitive > BootScriptExecutorDxe.efi.sensitive.txt
1 Module: BootScriptExecutorDxe.efi
2 FullPath: ..\\BIOS_Inspector_v2\\outputs\\modules\\BootScriptExecutorDxe.efi
3 Matches: 2
4
5 GUID  ASCII  0x9f40  0123456789ABCDEFf
6 STRING ASCII  0x9fab  Warning Write Failure
7

```

Figura 5. Arquivo module.efi.sensitive.txt com correspondências de strings e GUIDs

- Manifesto consolidado e relatórios em nível de firmware:** Além dos relatórios por módulo, o BIOS Inspector gera um manifest.json na raiz da pasta outputs. Esse arquivo consolida tudo que foi observado, módulo por módulo, incluindo metadados PE, seções, hashes, strings, correspondências sensíveis e status de execução da etapa dinâmica opcional. Do ponto de vista do usuário, o manifest funciona como uma visão de “tudo em um lugar”, que pode ser consumida posteriormente por outras ferramentas, scripts ou painéis, sem que seja necessário reprocessar os binários. Os relatórios em TXT e CSV, produzidos por meio das classes de acesso a dados, completam a experiência ao oferecer formatos amigáveis para inspeção manual e integração com planilhas ou outros sistemas de apoio à decisão em segurança de firmware.
- Execução dinâmica opcional com Qiling:** Quando o usuário habilita a opção de execução dinâmica, o BIOS Inspector passa a acionar o framework Qiling módulo a módulo, respeitando um tempo limite configurável. Os logs produzidos pelo Qiling são organizados

por módulo dentro da pasta outputs\qiling_logs. Assim, quem usa a ferramenta pode correlacionar rapidamente um módulo suspeito encontrado nos relatórios estáticos com o respectivo log de execução dinâmica. A Figura 6 mostra à esquerda um log da execução via Qiling para um dos módulos e, à direita, a disposição da pasta de output onde cada subpasta contém o resultado de cada módulo.

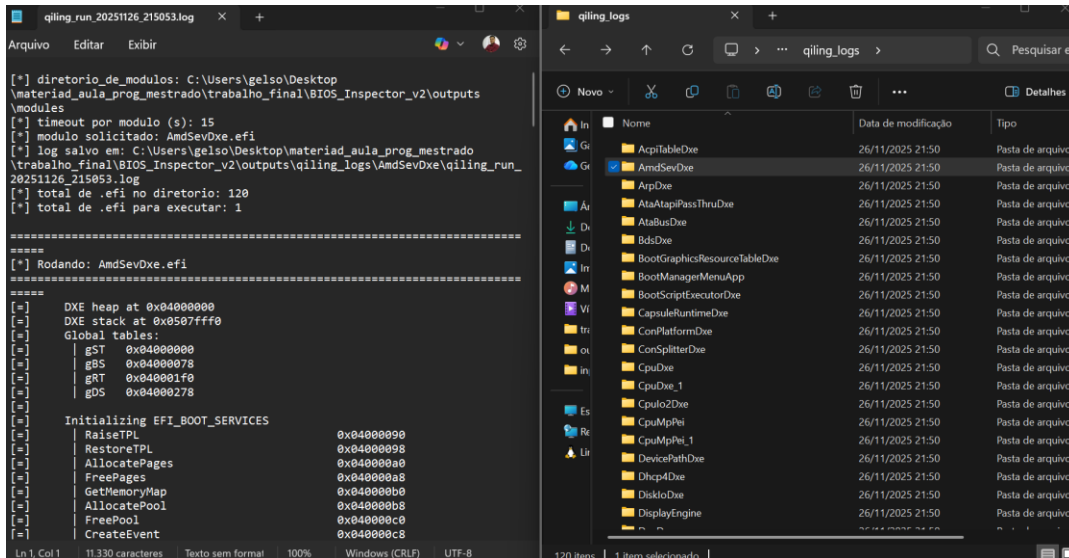


Figura 6. Log de execução via Qiling e organização da pasta de resultados

- **Operação via linha de comando e perfis de uso:** Toda a interação do usuário com o BIOS Inspector ocorre pela linha de comando. A ferramenta aceita parâmetros como caminho do firmware de entrada, pasta de workspace, número de threads, limites mínimos de tamanho de strings e ativação ou não da etapa Qiling. Isso permite desde um uso pontual em laboratório até a integração em pipelines de segurança mais amplos. A figura 7 mostra o “help” para compreender os parâmetros configuráveis na chamada do BIOS Inspector.

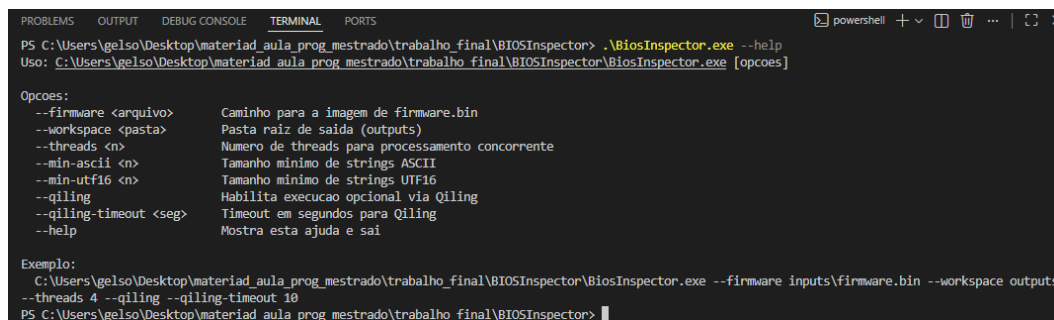


Figura 7. Parâmetros configuráveis para execução do BIOS Inspector

DESENVOLVIMENTO DO SOFTWARE NA VERSÃO ORIENTADA A OBJETOS

A Tabela 1 resume os requisitos funcionais definidos na proposta, com os respectivos pesos atribuídos pelo professor e o status de implementação na versão final do BIOS Inspector.

Tabela 1. Lista de Requisitos do BIOS Inspector.

N.	Requisitos Funcionais	Peso atribuído	Situação
1	Receber imagem de BIOS como entrada (arquivo único)	5	Realizado
2	Extrair módulos UEFI (.efi) por meio do CHIPSEC (uefi decode)	5	Realizado
3	Realizar fallback automático para UEFIEExtract quando necessário	5	Realizado ¹
4	Catalogar módulos extraídos (caminho, tamanho, data)	5	Semi realizado ²
5	Ler cabeçalho PE/COFF (campos básicos) e listar seções	5	Realizado
6	Calcular SHA-256 do arquivo e hashes por seção	5	Realizado
7	Extrair GUIDs presentes nos binários analisados	5	Realizado
8	Extrair strings ASCII/UTF-16 com tamanho mínimo configurável	5	Realizado
9	Destacar strings sensíveis conforme lista fornecida pelo usuário	5	Realizado
10	Persistir resultados em manifest.json (schema consistente)	10	Realizado
11	Exportar relatórios em TXT/CSV por meio do padrão DAO e oferecer filtros e buscas	10	Semi realizado ³
12	Interface por linha de comando com parâmetros configuráveis	10	Realizado
13	Processamento concorrente dos módulos (thread pool)	10	Realizado
14	Tratamento de erros por módulo, com continuidade do processamento	5	Realizado
15	Execução controlada opcional de módulos via Qiling, com timeout	5	Realizado
16	Registro de status de execução (sucesso, erro, timeout) e log curto, persistindo o log por meio do padrão DAO	10	Semi realizado ⁴

¹ Na versão final o UEFIEExtract é sempre executado após o CHIPSEC, sobrescrevendo ou complementando os módulos extraídos. Na prática o efeito de fallback automático é atendido, pois a segunda ferramenta corrige eventuais falhas da primeira.

² O sistema cataloga caminho e tamanho de cada módulo no Manifest, porém o campo de data de modificação ainda não é preenchido a partir do sistema de arquivos.

³ Os relatórios CSV e TXT são exportados por uma classe DAO dedicada, e seu formato foi pensado para permitir filtros e buscas em ferramentas como planilhas e scripts. Não há, contudo, uma camada de consulta interativa dentro do próprio BIOS Inspector.

⁴ O ExecutionStatus registra se o módulo foi executado ou não, o código de saída e um log curto, e estes dados são persistidos via ManifestDAO. A diferenciação explícita de timeout, separada do caso de erro genérico, depende hoje do código de retorno do script Python, portanto foi considerada parcialmente atendida.

Arquitetura orientada a objetos

O projeto foi estruturado em torno de um conjunto de classes que refletem bem a separação entre configuração, regras de negócio, infraestrutura e persistência.

No centro da modelagem está o tipo Config, que agrega todos os parâmetros da execução: caminhos de entrada e saída, localização dos scripts externos, quantidade de threads, limites mínimos de strings e timeout de Qiling. O Config é preenchido pela classe BiosInspectorApp, que faz o parsing da linha de comando e prepara a árvore de diretórios de trabalho.

A coleta de módulos UEFI foi encapsulada em duas classes que implementam a interface IModuleExtractor:

- ChipsecExtractor, responsável por invocar o script chipsec_util.py, mover os artefatos gerados e copiar os arquivos .efi para a pasta de módulos
- UefiExtractExtractor, responsável por executar o UEFIEExtract.exe, renomear os body.bin para <NomeModulo>.efi, copiar os módulos resultantes e organizar o dump e os relatórios

Do ponto de vista do usuário, isso aparece como as fases 1 e 2 do fluxo de execução, enquanto na visão de projeto essas classes isolam o código C++ das particularidades de cada ferramenta externa.

Em seguida, a classe PeCoffAnalyzer realiza a análise estática dos módulos. Ela preenche, para cada ModuleRecord, os metadados de arquivo e um objeto PeInfo, que contém campos do cabeçalho PE COFF, lista de seções, entropia e hashes SHA 256 de cada região de dados. A associação com GUIDs sensíveis é feita por meio de um mapa em memória construído a partir do arquivo <firmware>.guids.csv, originado pelo UEFIEExtract.

As classes `StringExtractor` e `SensitiveMatcher` implementam a parte de descoberta de artefatos de texto. A primeira percorre o binário de cada módulo em busca de sequências ASCII e UTF 16 acima de um tamanho mínimo configurável, preenchendo um vetor de `StringEntry` e persistindo um relatório de strings por módulo. A segunda utiliza listas de padrões sensíveis, armazenadas em arquivos texto, para gerar objetos `SensitiveMatch` tanto a partir das strings extraídas quanto a partir dos GUIDs associados a cada módulo. Os resultados são gravados em arquivos `.sensitive.txt` e também contabilizados nos relatórios de alto nível.

A execução dinâmica opcional é encapsulada na classe `QilingExecutor`. Essa classe monta a linha de comando do script Python `qilingExec.py`, direcionando a saída para uma pasta específica por módulo, e resume o resultado em um `ExecutionStatus`. O diagrama de sequência apresentado na Figura 9 mostra a interação entre `BiosInspectorApp`, `QilingExecutor` e o script externo durante essa fase.

Para coordenar o processamento em paralelo, foi implementado um `ThreadPool` simples baseado em `std::thread`, `std::mutex`, `std::condition_variable` e uma fila de tarefas. Cada `ModuleRecord` é enviado ao pool em uma tarefa que executa, em sequência, a extração de strings, o casamento de padrões sensíveis e, se habilitado, a execução sob Qiling. Isso reduz o tempo total de análise quando há dezenas ou centenas de módulos, respeitando o número de threads configurado na linha de comando.

Aplicação do padrão DAO e persistência

O padrão DAO foi aplicado explicitamente em duas classes: `ManifestDAO` e `ReportDAO`. Ambas atuam na camada de persistência, isto é, na forma como os dados do domínio são gravados em disco.

`ManifestDAO` recebe um objeto `Manifest`, que contém a lista completa de `ModuleRecord`, e escreve um arquivo `manifest.json` no diretório de trabalho. Esse JSON registra, para cada módulo, caminho, tamanho, informações PE COFF, strings extraídas, matches sensíveis e o `ExecutionStatus`. A lógica de serialização e o formato do arquivo permanecem encapsulados nesta classe, o que permite evoluir o schema sem impactar o restante do código.

`ReportDAO` é responsável por gerar relatórios derivados em formatos voltados ao uso operacional. Atualmente são produzidos um arquivo `modules_summary.csv`, adequado para filtros e buscas em planilhas, e um `modules_report.txt`, com resumo textual por módulo. Novos formatos, como relatórios específicos por área ou integrações com outras ferramentas, podem ser adicionados por meio de novos métodos nesta mesma classe ou em novas classes DAO.

Essa separação entre domínio e persistência facilita testes unitários, permite reutilizar a estrutura de dados em outros contextos, e atende diretamente aos requisitos 10, 11 e 16 com o uso explícito do padrão DAO.

Uma característica importante do BIOS Inspector é a integração com ferramentas especializadas escritas em outras linguagens. A classe ChipsecExtractor depende de um script Python oficial do projeto CHIPSEC, a classe UefiExtractExtractor invoca um executável em C para desmontar a imagem UEFI, e o QilingExecutor orquestra um script Python que utiliza o framework Qiling para emulação de módulos.

Essa integração exigiu cuidados com montagem de comandos, tratamento de caminhos em Windows e captura de códigos de retorno. As funções auxiliares runCommand e run_command_streaming centralizam essa preocupação, permitindo logar o comando executado e reagir a falhas sem interromper o processamento dos demais módulos. Dentro do ThreadPool, exceções são capturadas e convertidas em mensagens de log, garantindo que um erro pontual em um módulo não derrube toda a execução, o que reforça o atendimento do requisito R14.

No conjunto, essa seção mostra que o desenvolvimento do BIOS Inspector explorou conceitos centrais de orientação a objetos, uso da STL, concorrência, padrão DAO e integração com scripts externos, tudo alinhado aos requisitos definidos para o projeto da disciplina de Programação Avançada. O projeto está público no Github e disponível em 26/11/2025 no link: <https://github.com/GelsonFilho/BIOSInspector>. Os diagramas UML (básico e completo) e diagrama de sequência podem ser visualizados na pasta “diagrams” do projeto.

TABELA DE CONCEITOS UTILIZADOS E NÃO UTILIZADOS

Tabela 2. Tabela de conceitos utilizados e não utilizados

N.	Conceitos	Uso	Onde
1	Elementares: classes e objetos; atributos privados; métodos com e sem retorno; construtores e destrutores; classe principal; divisão em .h e .cpp	Sim	Todas as classes em BiosInspector.h/BiosInspector.cpp; função main instanciando BiosInspectorApp
2	Relações: associação, agregação, herança simples; herança em vários níveis e múltipla (não utilizadas)	Parcial	Associação e agregação entre BiosInspectorApp, Config, ModuleRecord, Manifest; herança de IModuleExtractor para ChipsecExtractor e UefiExtractExtractor
3	Ponteiros, generalizações, exceções, templates próprios (apenas exceções utilizadas)	Parcial	Blocos try/catch em ChipsecExtractor::extract, UefiExtractExtractor::extract e outras rotinas; não há uso de new/delete nem templates próprios

4	Sobrecarga de construtores/métodos; sobrecarga de operadores; persistência texto/binário (operadores e binário não utilizados)	Parcial	Construtores e métodos com parâmetros diferentes em várias classes; persistência texto em ManifestDAO::save e ReportDAO::exportReports; sem sobrecarga de operadores nem arquivos binários
5	Virtualidade: métodos virtuais; polimorfismo; métodos virtuais puros; coesão e desacoplamento	Sim	Interface abstrata IModuleExtractor com método virtual puro extract; polimorfismo entre extratores; separação entre classes de domínio, serviço e DAO
6	Engenharia de software: requisitos textuais e tabelados; diagrama de classes; outros diagramas UML	Sim	Requisitos descritos e tabelados no relatório; uso de Diagrama de Classes, versão simplificada e Diagrama de Sequência
7	Biblioteca gráfica; interdisciplinaridade (apenas interdisciplinaridade utilizada)	Parcial	Sem biblioteca gráfica; uso de entropia de Shannon e conceitos de arquitetura/segurança de firmware
8	Organizadores: namespaces próprios; classes aninhadas; membros estáticos; uso de std::string (apenas std::string utilizado)	Parcial	Namespace biosinspector e uso intensivo de std::string; sem classes aninhadas nem membros estáticos relevantes
9	STL: std::vector; std::list; pilhas/filas/conjuntos/mapas (vetor e mapas utilizados)	Parcial	std::vector para listas de módulos, seções, strings, padrões, threads; std::unordered_map em load_guid_map; sem list, queue, set etc.
10	Conceitos avançados: padrões de projeto, programação concorrente, API de rede, GUI (concorrência e DAO utilizados)	Parcial	ThreadPool com std::thread, std::mutex, std::condition_variable e fila de tarefas; padrão DAO em ManifestDAO e ReportDAO; sem rede nem GUI

Tabela 3. Lista de Justificativas para Conceitos Utilizados e Não Utilizados no Trabalho

N	Conceitos	Justificativa
1	Elementares (classes, objetos, atributos, métodos, construtores, destrutores, .h/.cpp)	Base de todo o projeto, usadas em todas as classes e arquivos para organizar o domínio BIOS Inspector.
2	Relações (associação, agregação, herança simples via IModuleExtractor)	Necessárias para compor objetos como Manifest, Config e estratégias de extração sem acoplamento forte.
3	Exceções e uso de STL em vez de new/delete	Exceções sinalizam falhas críticas e a STL evita gerenciamento manual de memória e erros de ponteiro.
4	Sobrecarga de construtores e métodos simples	Usada apenas onde ajuda na configuração de objetos como Config e ThreadPool, sem sobrecarregar operadores.
5	Virtualidade e polimorfismo (IModuleExtractor)	Permite trocar ChipsecExtractor e UefiExtractExtractor sem mudar o código cliente, facilitando extensão futura.
6	Engenharia de software (requisitos, UML, diagrama de sequência)	Guiaram a implementação e documentam o sistema de forma alinhada com a disciplina.
7	Biblioteca gráfica	Não usada porque a ferramenta é de linha de comando e voltada para automação e pipelines de segurança.
8	Organizadores (namespace, funções auxiliares, std::string)	Namespace biosinspector agrupa o domínio e std::string simplifica o tratamento de textos e caminhos.
9	STL (vector, queue, unordered_map, optional)	Usados para listas de módulos, fila de tarefas, mapa de GUIDs e metadados opcionais, com boa performance.
10	Conceito avançado (programação concorrente com ThreadPool)	Threads e sincronização reduzem o tempo de análise em firmwares grandes e atendem ao requisito de conceito avançado.

DISCUSSÃO E CONCLUSÃO

O projeto BIOS Inspector atendeu ao objetivo proposto na disciplina de Programação Avançada, pois consolidou conceitos de orientação a objetos em C++ por meio de um sistema realista, ligado ao contexto de segurança de firmware. A solução resultante automatiza uma sequência que antes era fragmentada: extração de módulos UEFI, análise PE COFF, coleta de

strings e busca por padrões sensíveis, além do registro estruturado em manifest JSON e relatórios em TXT e CSV.

Do ponto de vista técnico, destacam-se a separação clara entre domínio e persistência, o uso de DAO para geração de manifest e relatórios, a aplicação de concorrência com um thread pool simples e a integração com ferramentas externas como CHIPSEC, UEFIEExtract e Qiling por meio de chamadas de sistema controladas. Esses elementos mostram o uso prático de herança, polimorfismo, STL, exceções e programação concorrente em um contexto próximo ao mercado.

O projeto apresenta também limitações. A ferramenta depende de executáveis externos específicos e atualmente está focada em ambiente Windows, o que restringe a portabilidade. A fase dinâmica com Qiling ainda é opcional e exploratória, sem cobertura completa de todos os fluxos possíveis dos módulos UEFI. Além disso, a interface é exclusivamente de linha de comando, sem camada gráfica ou integração direta com outros sistemas corporativos.

Como trabalhos futuros, vislumbra-se a evolução do BIOS Inspector para suportar outros formatos de firmware e plataformas, a ampliação do conjunto de métricas estáticas coletadas, a integração com regras de detecção mais avançadas e uma camada de orquestração que permita encaixar a ferramenta em pipelines DevSecOps de forma automatizada. Mesmo com essas limitações, o resultado já se mostra útil como ferramenta de apoio inicial à triagem e à revisão de segurança de BIOS UEFI em escala.

REFERENCIAS

[1] QILING FRAMEWORK. Qiling: Binary emulation framework.

Disponível em: <<https://github.com/qilingframework/qiling>>. Acesso em: 26/11/2025.

[2] CHIPSEC. Platform Security Assessment Framework.

Disponível em: <<https://github.com/chipsec/chipsec>>. Acesso em: 26/11/2025.

[3] LONG, C. UEFITool e UEFIEExtract.

Disponível em: <<https://github.com/LongSoft/UEFITool>>. Acesso em: 26/11/2025.

[4] UNIFIED EFI FORUM. Unified Extensible Firmware Interface (UEFI) Specification.

Disponível em: <<https://uefi.org/specifications>>. Acesso em: 26/11/2025.

[5] MICROSOFT. PE and COFF Specification.

Disponível em: <<https://learn.microsoft.com/en-us/windows/win32/debug/pe-format>>. Acesso em: 26/11/2025.

[6] ISO. ISO/IEC 14882:2020 – Programming Language C++.

Informações gerais disponíveis em: <<https://www.iso.org/standard/79358.html>>. Acesso em: 26/11/2025.

